

Lecture 2 : Types, Operators, and Expressions in C

In C programming, data is managed through variables and constants.

To understand how a program manipulates data, we look at four core concepts:

- * Variables: Named storage locations for data.

- * Constants: Fixed values that do not change.

- * Operators: Symbols that specify the action to be performed (e.g., +, *).

- * Expressions: Combinations of variables and constants that result in a new value.

1. Identifiers (Naming Rules)

An identifier is the name you give to variables, functions, or macros.

Technical Rules:

- * Characters: Must consist of letters and digits.



- * Starting Character: Must begin with a letter or an underscore (_). It cannot start with a digit.

- * Case Sensitivity: C is case-sensitive (Total and total are two different identifiers).

- * Keywords: There are 32 reserved keywords (like int, while, return) that cannot be used as names.

Best Practices (Style Guide):

- * Variables: Use lowercase (e.g., count).

- * Constants: Use uppercase (e.g., MAX_VALUE).

- * Length: Local variables should be short/simple; external/global names should be descriptive.

- * Caution: Avoid starting names with _ as they are often reserved for system libraries.



Data Types

C is a typed language, meaning every variable must have a declared type. This allows the compiler to allocate memory correctly and catch errors early.

The Three Basic Types:

- * Characters (char): Used for single letters or small integers (stored as a single byte).

- * Integers (int): Used for whole numbers.

- * Floating-point (float / double): Used for numbers with decimal points.

> Why types matter: A type defines the set of values an object can hold and the operations (math, logic) that can be legally performed on it.

Identifier Limits (Standard vs. Practice)

* Internal Names:

The ISO standard requires compilers to recognize at least the first 31 characters (case-sensitive).

* External Names:

(Used across different files) These are more restricted.

Historically, some systems only recognized the first 6 characters and ignored case sensitivity. However, most modern implementations are much more flexible.



C Data Types	
char	usually 8-bits (1 byte)
int	usually the natural word size for a machine or OS (e.g., 16, 32, 64 bits)
short int	at least 16-bits
long int	at least 32-bits
float	usually 32-bits
double	usually 64-bits
long double	usually at least 64-bits

Table 2.1: C data types and their usual sizes.

```
#include <stdio.h>
```

```
int main() {
```

```
1 //  . Declarations: Giving variables a type  
and a name
```

```
    int students = 150;    // Integer type for  
whole numbers
```

```
    float average = 72.45; // Floating-point  
for decimals
```

```
    char grade = 'B';    // Character type  
for single letters
```

```
2 //  . Printing the values to the screen
```

```
    printf("University Course Details:\n");
```

```
    printf("-----\n");
```

```
    // %d prints the integer value
```

```
    printf("Number of Students: %d\n",  
students);
```



```
printf("Class Average:    %f\n", average);

// %c prints the single character
printf("Required Grade:   %c\n", grade);

return 0;
}
```

The Output (What appears on the screen)

When the program executes, the terminal or console will display the following:

University Course Details:

Number of Students: 150

Class Average: 72.45

Required Grade: B

Constants in C

Constants are fixed values that do not change during program execution. In C, constants can be represented in different types and number systems.

Integer Constants & Suffixes

The type of an integer constant is determined by its value and any optional suffixes:

int: A standard number like 1234 is treated as a basic integer.

long: Suffix with L or l (e.g., 1234L). Note that numbers too large for a standard int are automatically treated as long.

unsigned: Suffix with U or u (e.g., 1234U).

unsigned long: Suffix with UL or ul (e.g., 1234UL).

Number Systems (Bases)

C allows you to write the same value in three different mathematical bases:

Decimal (Base 10): Written normally (e.g., 1234).

Octal (Base 8): Must start with a leading 0 (e.g., 02322).

Hexadecimal (Base 16): Must start with 0x or 0X (e.g., 0x4D2).

Key Point: Whether you write 1234, 02322, or 0x4D2, the computer stores them as the exact same binary value in memory.

Code Example & Output

Here is how these different representations work in a real program.

```
#include <stdio.h>

int main() {

    // These three variables hold the exact
    same value

    int x = 1234; // Decimal
    int y = 02322; // Octal
    int z = 0x4D2; // Hexadecimal

    // %d = decimal, %o = octal, %x = hex
    printf("Different formats of X:
    %d\t%o\t%x\n", x, x, x);

    // Printing all three as decimals to prove
    they are equal

    printf("Verification: %d\t%d\t%d\n", x, y,
    z);

    return 0;
}
```

The Output on Screen:

Different formats of X: 1234 2322 4d2

Verification: 1234 1234 1234



Symbolic Constants

Symbolic constants allow you to give a meaningful name to a constant value.

This practice is essential for writing professional, maintainable code.

The #define Directive

In C, symbolic constants are created using the #define preprocessor command.

Syntax: #define NAME value

Mechanism: The compiler performs a direct text-replacement. Wherever the "NAME" appears in your code, it is replaced by the "value" before the program runs.

```
#define BLOCK_SIZE 100          // Integer  
constant
```

```
#define HELLO "Hello World\n"   // String  
constant
```

```
#define EXP 2.7183              // Floating-  
point constant
```

Code Example & Output

```
#include <stdio.h>

#define PI 3.14159

#define WELCOME "Welcome to\nEngineering Class\n"

int main() {

    float radius = 5.0;

    float area = PI * radius * radius; // PI is
replaced by 3.14159

    printf(WELCOME); // WELCOME is
replaced by the string

    printf("Area of the circle: %f\n", area);

    return 0;
}
```

The Output on Screen:

Welcome to Engineering Class

Area of the circle: 78.54

printf() Conversion Specifiers

The printf() function is used to produce formatted output by merging numerical or character values into a string.

It uses conversion specifiers as placeholders for variables.

Common Conversion Specifiers

To display data correctly, you must use the symbol that matches the data type:

%c: Used for single characters (e.g., 'a', 'b').

%f: Used for floating-point numbers in decimal notation.

%e: Used for scientific notation (e.g., 2.000000e+03).

%s: Used for strings (text).

%d: Used for integers (decimal).



Critical Warning: Type Matching

A conversion specifier must match the type of the variable being passed.

If there is a mismatch, the program will either print garbage values or crash.

Error Example: `printf("%f", 52);`

This is a mismatch because `%f` expects a float, but 52 is an integer.



Declarations

All variables must be declared before they are used.

They must be declared at the top of a block, a section of code enclosed in brackets { and }, before any statements.

They may be initialized by a constant or an expression when declared.

The following are a set of example declarations.

```
int lower, upper, step;
```

```
char buf[10]; /* an uninitialised array of  
chars */
```

```
float limit = 9.34f;
```

```
const double PI = 3.1416;
```


Arithmetic Operations in C

Arithmetic operators allow you to perform mathematical calculations.

They are divided into two categories: Binary (using two values) and Unary (using one value).

Binary Operators

Binary operators perform operations between two operands.

Addition (+), Subtraction (-), Multiplication (*): Work normally on both integers and floating-point numbers.

Division (/):

For floating-point numbers, it performs standard division.

Crucial Note:

For integers, division truncates (discards) the fractional part (e.g., $17 / 5$ equals 3).

Modulus (%): Returns the remainder of an integer division (e.g., $18 \% 7$ equals 4).

Constraint:

This operator is valid only for non-floating-point types like int and char.

Unary Operators

Unary operators act on a single variable or value.

Unary Plus (+) and Minus (-):

Used to indicate sign (e.g., -56.3).

The unary + is mostly redundant since numbers are positive by default.

Increment (++) and Decrement (--):

These add or subtract 1 from a variable.

$x++$ is shorthand for $x = x + 1$.

Prefix vs. Postfix (The Increment Difference)

The ++ and -- operators can be placed before or after a variable with different results:

Prefix (++x): The value is incremented before it is used in the expression.

Postfix (x++): The value is used first, then incremented afterward.

Relational and Logical Operations

Relational and logical operators are used to make decisions in a program.

They allow us to compare values and combine multiple conditions.

Relational Operators

These operators compare two values.

In C, a relational expression results in 1 (TRUE) or 0 (FALSE).

Example	Meaning	Operator
$5 > 3$	Greater than	$>$
$2.1 < 7$	Less than	$<$
$3 \geq 3$	Greater than or equal to	$\geq$
$4 \leq 2$	Less than or equal to	$\leq$
$x == x$	Equal to	$==$
$x != x$	Not equal to	$!=$

Critical Warning: Do not confuse = (assignment) with == (equality).

= (Assignment Operator):

It sets the value. It takes the value on the right and "forces" it into the variable on the left. eg `x= 3`.

== (Equality Operator):

It checks the value. It asks a question: "Are these two sides equal?" and answers with 1 (True) or 0 (False). eg `x==3`.

Logical Operators

These operators are used to combine multiple relational expressions.

&& (AND): TRUE only if both sides are TRUE.

|| (OR): TRUE if at least one side is TRUE.

! (NOT): A unary operator that reverses the truth value (TRUE becomes FALSE).

Example (where $a=1$, $b=2$, $c=3$, $d=3$):

$(a < b \ \&\& \ b < c \ \&\& \ c < d)$ is 0 (FALSE)
because $3 < 3$ is false.

$(a < b \ \&\& \ b < c \ \&\& \ c \leq d)$ is 1 (TRUE)
because all conditions are met."